ZyntraPay

High-Level Design Document

Digital Wallet & Rewards Platform

.NET 8 Microservices Architecture

Version 1.0 | 2026

Table of Contents

Table of Contents 2
1. Document Overview 4
1.1 Purpose 4
1.2 Scope 4
1.3 Audience 4
1.4 Assumptions and Constraints 4
2. System Overview 5
2.1 Business Goal 5
2.2 Problem Statement 5
2.3 Solution Summary 5
3. Architecture Overview 6
3.1 High-Level Architecture Description 6
3.2 High-Level Architecture Diagram 6
3.3 Microservices Architecture 6
3.4 Gateway Pattern 7
3.5 Database-Per-Service Pattern 7
3.6 Event-Driven Communication 7
3.7 Frontend-to-Backend Flow 7
4. Major Components 8
4.1 ApiGateway 8
4.2 AuthService 8
4.3 UserService 8
4.4 WalletService 8
4.5 RewardsService 8
4.6 NotificationService 8
4.7 AdminService 8
5. Functional Scope 9
5.1 Authentication and Authorization 9
5.2 User Profile and KYC 9
5.3 Wallet Management 9
5.4 Rewards and Redemption 9
5.5 Notifications 9
5.6 Admin Operations 10
6. API Architecture 10

6.1 Gateway Route Strategy 10
6.2 Service Endpoints Summary 10
AuthService APIs 10
UserService APIs 11
WalletService APIs 11
RewardsService APIs 11
NotificationService APIs 12
AdminService APIs 12
6.3 Token Flow 12
7. Data Architecture 12
7.1 Database Ownership 12
7.2 Cross-Service Identity Reference 13
7.3 Table-Level Summary 13
7.4 Caching Usage 15
8. Integration Architecture 16
8.1 RabbitMQ Event Flow 16
8.2 Inter-Service HTTP Calls 16
8.3 Downstream Service Communication 16
9. Security Architecture 17
9.1 JWT Authentication 17
9.2 Role-Based Authorization 17
9.3 Password Hashing 17
9.4 Token Forwarding 17
9.5 CORS Policy 17
9.6 Authentication Flow Diagram 18
10. Non-Functional Requirements 18
11. Deployment View 19
11.1 Service Ports 19
11.2 Runtime Dependencies 19
12. Current Implementation Status 19
12.1 Completed Backend Services 19
12.2 Remaining Work 20
12.3 Angular Integration Scope 20
13. Risks and Limitations 20
14. Conclusion 21

1. Document Overview

1.1 Purpose

This High-Level Design (HLD) document defines the overall system architecture, component interactions, data ownership, security model, and deployment strategy for ZyntraPay — a .NET 8 microservices-based digital wallet and rewards platform. It is intended to serve as the authoritative architectural reference for development, review, and onboarding activities.

1.2 Scope

This document covers the back-end microservices architecture, API gateway strategy, event-driven communication, database design per service, and the planned Angular frontend integration. It does not cover infrastructure provisioning, CI/CD pipeline details, or third-party payment gateway integration unless explicitly present in the codebase.

1.3 Audience

- Software architects and senior developers
- Backend and frontend engineers working on ZyntraPay
- QA engineers designing integration test strategies
- Technical reviewers and project stakeholders

1.4 Assumptions and Constraints

- The system is built on .NET 8 with ASP.NET Core Web API.
- Each microservice owns its own SQL Server database; no cross-service direct DB access is permitted.
- RabbitMQ is the message broker for all asynchronous communication.
- The Angular frontend is planned/in progress and communicates exclusively through ApiGateway.
- JWT Bearer tokens are the sole authentication mechanism for secured endpoints.
- Ocelot is the API Gateway library in use.
- BCrypt is used for all password hashing.
- In-memory caching (IMemoryCache) is used for tokens, OTPs, and hot-read data.

2. System Overview

2.1 Business Goal

ZyntraPay aims to provide users with a secure, scalable digital wallet that supports peer-to-peer transfers, wallet top-ups, and a loyalty rewards program. The platform enables administrators to perform KYC governance, user management, and operational oversight through a dedicated admin interface.

2.2 Problem Statement

Traditional financial applications are monolithic, making it difficult to independently scale individual capabilities such as authentication, wallet operations, or notifications. ZyntraPay addresses this by decomposing these concerns into discrete, independently deployable microservices that communicate through well-defined contracts.

2.3 Solution Summary

ZyntraPay is implemented as a set of seven loosely coupled microservices, each owning its own data store and exposing RESTful APIs. An Ocelot-based ApiGateway acts as the sole public entry point. Asynchronous integration is handled through RabbitMQ events. JWT tokens carry identity claims that propagate across service boundaries.

2.3 Role and Access Control

Role	Access Level	Primary Responsibilities
User	JWT Role = User	Register, login, manage wallet, submit KYC, earn and redeem rewards, view transactions
Admin	JWT Role = Admin	Approve/reject KYC, manage users, configure reward campaigns, view dashboard KPIs

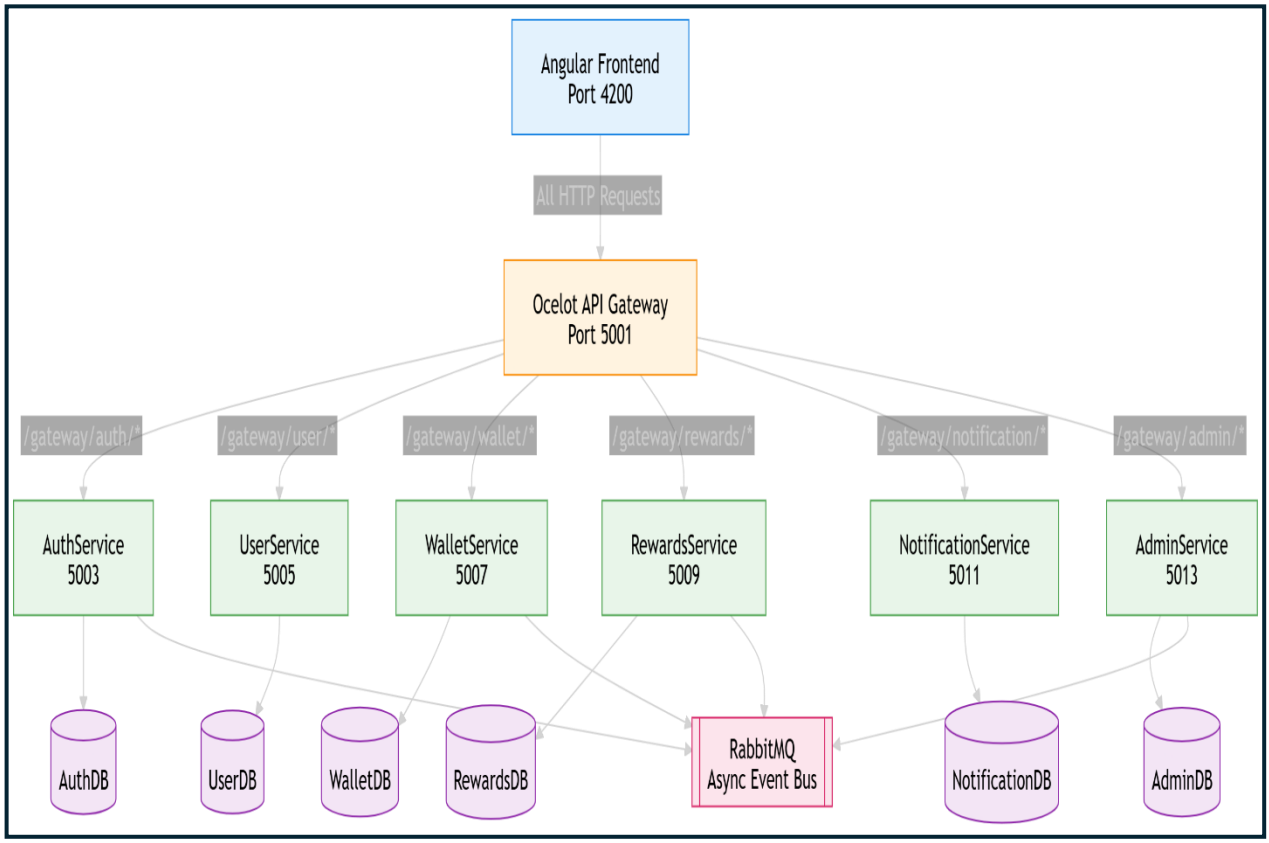
2.4 Technology Stack

Layer	Technology
Backend	.NET 8 Web API — Microservices architecture
Frontend	Angular 17 — Component-based SPA
Database	SQL Server (SSMS) with Entity Framework Core 8 — DB First
Authentication	JWT Bearer Tokens — role-based and claims-based
Messaging	RabbitMQ — async event-driven communication
API Gateway	Ocelot — centralized routing, CORS, single entry point
Password Hashing	BCrypt.Net — salted hash, no plain-text storage
API Documentation	Swagger / Swashbuckle — interactive API explorer per service
Logging	ILogger<T> — structured logging built into ASP.NET Core
Exception Handling	Global Exception Middleware — centralized error response

3. Architecture Overview

3.1 High-Level Architecture Description

ZyntraPay adopts a microservices architecture where each service is responsible for a single bounded context. Services expose REST APIs, are secured via JWT, and interact asynchronously through RabbitMQ. All client traffic is routed through the ApiGateway, which applies CORS policies and forwards requests with the original Authorization header intact.



3.2 High-Level Architecture Diagram

3.3 Microservices Architecture

Each service is an independent ASP.NET Core Web API project. Services are:

- Independently deployable and runnable on separate ports.
- Isolated at the data layer — no service accesses another service's database.
- Identified by a shared AuthUserId (GUID) that serves as the cross-service user key.
- Documented via Swagger UI at their respective ports.

3.4 Gateway Pattern

ApiGateway uses Ocelot to define upstream/downstream route mappings in ocelot.json. It does not contain business logic. It strips or forwards Authorization headers as configured per route, and applies CORS to permit requests from http://localhost:4200.

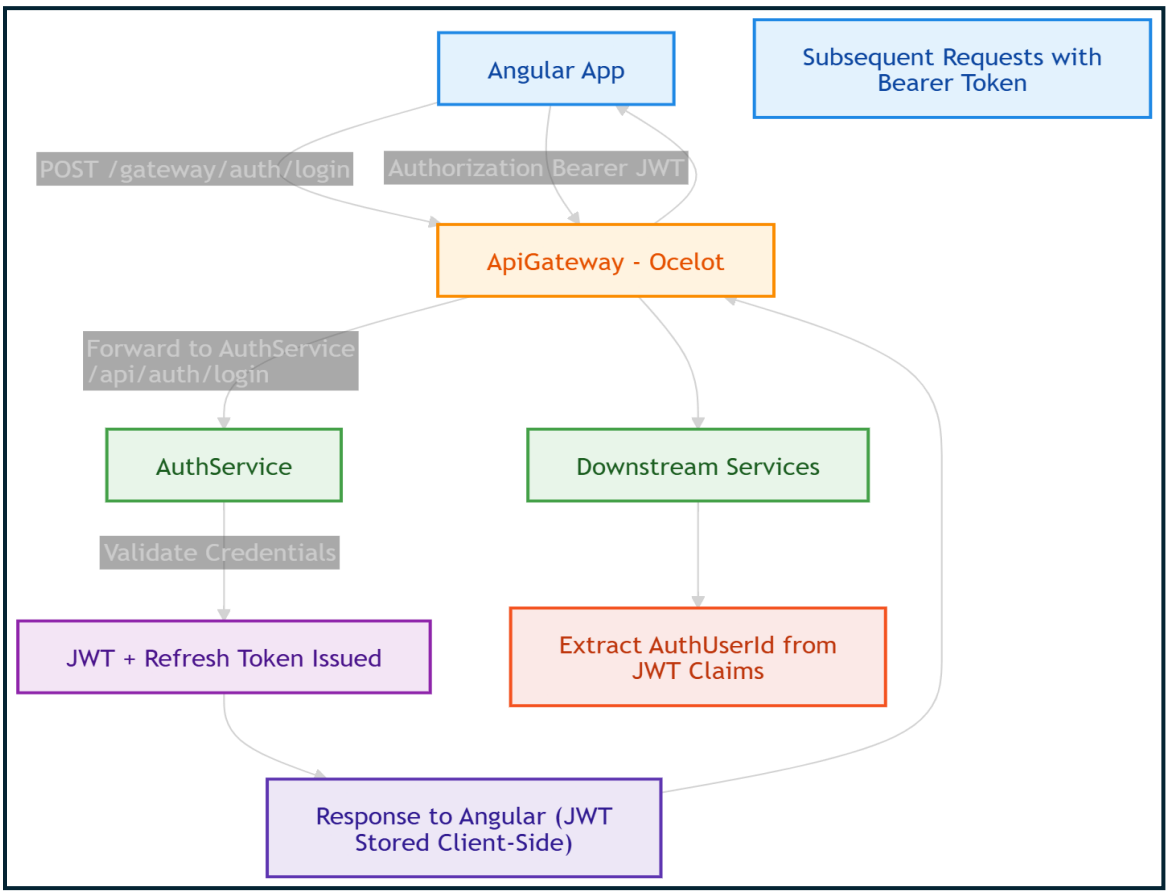
3.5 Database-Per-Service Pattern

Each microservice owns its own SQL Server database. Cross-service data access is performed only through HTTP calls or events. The AuthUserId column serves as a logical foreign key in all service databases, referencing AuthDB.Users.Id.

3.6 Event-Driven Communication

RabbitMQ is used for asynchronous event propagation. After completing monetary operations, the WalletService publishes events that are consumed by RewardsService and NotificationService. AuthService publishes OTP and welcome email events consumed by NotificationService.

3.7 Frontend-to-Backend Flow



4. Major Components

4.1 ApiGateway

Technology: ASP.NET Core + Ocelot. Port: 5000. Responsibilities: route forwarding, CORS configuration for Angular, token passthrough. Contains no business logic. All public client requests must pass through this gateway.

4.2 AuthService

Technology: ASP.NET Core Web API, EF Core, SQL Server, BCrypt, IMemoryCache. Port: 5001. Manages the Users table in AuthDB. Handles registration (user and admin), login, Google login, OTP generation/verification, forgot/reset password, refresh token management, phone number update, and admin user management endpoints.

4.3 UserService

Technology: ASP.NET Core Web API, EF Core, SQL Server. Port: 5002. Manages UserProfiles and KycSubmissions tables in UserDB. Handles profile creation, profile retrieval, KYC submission, KYC status inquiry, and KYC review workflows.

4.4 WalletService

Technology: ASP.NET Core Web API, EF Core, SQL Server, IMemoryCache, RabbitMQ publisher. Port: 5003. Manages Wallets and LedgerEntries tables in WalletDB. Handles wallet creation, balance retrieval (with caching), top-up, peer-to-peer transfer, and transaction history.

4.5 RewardsService

Technology: ASP.NET Core Web API, EF Core, SQL Server, IMemoryCache, RabbitMQ consumer. Port: 5004. Manages RewardAccounts, RewardCatalog, and Redemptions tables in RewardsDB. Handles reward account summaries, catalog browsing, point redemptions, redemption history, and automatic point awards triggered by wallet events.

4.6 NotificationService

Technology: ASP.NET Core Web API, EF Core, SQL Server, RabbitMQ consumer. Port: 5005. Manages the Notifications table in NotificationDB. Stores notification records, retrieves notifications per user, and marks notifications as read. Notification records are created by event consumers reacting to system events.

4.7 AdminService

Technology: ASP.NET Core Web API, EF Core, SQL Server, typed HttpClient, AuthTokenHandler. Port: 5006. Manages AdminActions table in AdminDB. Provides admin-only endpoints for pending KYC review, KYC approval/rejection, user listing, user status toggle, and dashboard statistics. Calls AuthService and UserService via typed HttpClients with forwarded JWT.

5. Functional Scope

5.1 Authentication and Authorization

- User registration with OTP-verified email
- Admin registration with secret key validation
- Login with email/password (BCrypt verification)
- Google OAuth login (ID token validation)
- JWT issuance containing NameIdentifier, Email, and Role claims
- Refresh token generation and rotation (stored in IMemoryCache)
- OTP request and verification (10-minute cache window)
- Forgot password and reset password via OTP
- Phone number update for authenticated users
- Admin-only user list and user active/inactive toggle

5.2 User Profile and KYC

- Create user profile (one per AuthUserId)
- Retrieve own profile
- Submit KYC documents (one submission per user)
- View own KYC status (Pending / Approved / Rejected)
- Admin retrieval of all pending KYC submissions
- Admin KYC review with Approved or Rejected decision; rejection requires a reason

5.3 Wallet Management

- Create wallet (one per AuthUserId, starts with zero balance)
- Retrieve wallet balance (with short-duration cache)
- Top-up wallet (creates CREDIT ledger entry, updates balance, publishes event)
- Transfer between wallets (creates DEBIT and CREDIT ledger entries, updates both balances, publishes event)
- Transaction history retrieval (all ledger entries for the wallet)
- Individual transaction lookup by ID

5.4 Rewards and Redemption

- Reward account summary (total points and current tier)
- Reward catalog browsing (active items only, cached for 5 minutes)
- Redeem catalog item (points deducted, stock reduced, tier recalculated)
- Redemption history per user
- Automatic point award on wallet top-up events (account auto-created if missing)

5.5 Notifications

- Store notification records generated by system events

- Retrieve all notifications for the authenticated user
- Mark a specific notification as read (owner-only)
- Event consumers react to wallet and KYC events to create notifications

5.6 Admin Operations

- Retrieve pending KYC submissions (proxied from UserService)
- Review and approve/reject KYC (proxied to UserService, action logged)
- Retrieve all users (proxied from AuthService)
- Toggle user active/inactive status (proxied to AuthService, action logged)
- Dashboard statistics (assembled from downstream service data)
- All admin actions persisted in AdminActions audit table

6. API Architecture

6.1 Gateway Route Strategy

ApiGateway forwards all requests prefixed with /gateway/{service}/* to the corresponding downstream service. The JWT Authorization header is forwarded transparently, allowing each downstream service to validate the token independently.

Gateway Prefix	Downstream Service	Port
/gateway/auth/*	AuthService	5001
/gateway/user/*	UserService	5002
/gateway/wallet/*	WalletService	5003
/gateway/rewards/*	RewardsService	5004
/gateway/notification/*	NotificationService	5005
/gateway/admin/*	AdminService	5006

6.2 Service Endpoints Summary

AuthService APIs

Method	Endpoint	Auth	Description
POST	/api/auth/register	None	User registration
POST	/api/auth/register-admin	None	Admin registration with secret key
POST	/api/auth/login	None	Email/password login
POST	/api/auth/google-login	None	Google OAuth login

Method	Endpoint	Auth	Description
PUT	/api/auth/update-phone	JWT	Update phone number
POST	/api/auth/send-otp	None	Request OTP
POST	/api/auth/verify-otp	None	Verify OTP
POST	/api/auth/forgot-password	None	Request password reset OTP
POST	/api/auth/reset-password	None	Reset password with OTP
POST	/api/auth/refresh-token	None	Obtain new JWT/refresh pair
GET	/api/auth/admin/users	Admin JWT	List all users
PUT	/api/auth/admin/users/{id}/toggle	Admin JWT	Toggle user active status
GET	/api/auth/users/{authUserId}/email	Internal	Fetch email by auth user ID

UserService APIs

Method	Endpoint	Auth	Description
POST	/api/user/profile	JWT	Create user profile
GET	/api/user/profile	JWT	Get own profile
POST	/api/user/kyc	JWT	Submit KYC documents
GET	/api/user/kyc	JWT	Get own KYC status
GET	/api/user/admin/kyc/pending	Admin JWT	List pending KYC submissions
PUT	/api/user/admin/kyc/{kycId}/review	Admin JWT	Review KYC (approve/reject)

WalletService APIs

Method	Endpoint	Auth	Description
POST	/api/wallet/create	JWT	Create wallet
GET	/api/wallet/balance	JWT	Get wallet balance (cached)
POST	/api/wallet/topup	JWT	Top-up wallet balance
POST	/api/wallet/transfer	JWT	Transfer to another wallet
GET	/api/wallet/transactions	JWT	List all transactions
GET	/api/wallet/transactions/{id}	JWT	Get single transaction

RewardsService APIs

Method	Endpoint	Auth	Description
GET	/api/rewards/summary	JWT	Get reward account summary

Method	Endpoint	Auth	Description
GET	/api/rewards/catalog	JWT	Get reward catalog (cached)
POST	/api/rewards/redeem	JWT	Redeem catalog item
GET	/api/rewards/history	JWT	Get redemption history

NotificationService APIs

Method	Endpoint	Auth	Description
GET	/api/notification	JWT	Get notifications for current user
PUT	/api/notification/{id}/read	JWT	Mark notification as read

AdminService APIs

Method	Endpoint	Auth	Description
GET	/api/admin/kyc/pending	Admin JWT	Get pending KYC list
PUT	/api/admin/kyc/{kycId}/review	Admin JWT	Review KYC
GET	/api/admin/users	Admin JWT	Get all users
PUT	/api/admin/users/{userId}/toggle	Admin JWT	Toggle user status
GET	/api/admin/dashboard	Admin JWT	Get dashboard statistics

6.3 Token Flow

1. Client obtains JWT from AuthService via /api/auth/login or /api/auth/google-login.
2. Client includes the JWT in the Authorization: Bearer <token> header on subsequent requests.
3. ApiGateway forwards the header downstream without modification.
4. Each service validates the JWT using the shared signing key and extracts the AuthUserId from the NameIdentifier claim.
5. Admin endpoints additionally require the Role claim to equal Admin.
6. AdminService forwards the token to downstream services using AuthTokenHandler.

7. Data Architecture

7.1 Database Ownership

Service	Database	Primary Tables
AuthService	AuthDB	Users
UserService	UserDB	UserProfiles, KycSubmissions
WalletService	WalletDB	Wallets, LedgerEntries
RewardsService	RewardsDB	RewardAccounts, RewardCatalog, Redemptions
NotificationService	NotificationDB	Notifications
AdminService	AdminDB	AdminActions

7.2 Cross-Service Identity Reference

All services share the AuthUserId (GUID) as the cross-service user identifier. This value is stored as a column in each service's tables and corresponds logically to AuthDB.Users.Id. There is no database-level foreign key across service databases.

7.3 Table-Level Summary

1. Authentication Service (AuthDB)

Purpose: Handles identity, credentials, and access control.

Field Name	Data Type / Constraints	Description
Id	GUID (PK)	Unique identifier for the user across all services.
Email	Unique, Varchar(150)	User's primary login identifier.
PhoneNumber	Unique, Varchar(20)	User's contact number for MFA/Alerts.
PasswordHash	Varchar(300)	Securely hashed password string.
Role	String (Default: 'User')	RBAC (e.g., User, Admin, SuperAdmin).
IsActive	Boolean (Default: true)	Soft delete or account suspension status.
CreatedAt	DateTime (Default: GETDATE())	Timestamp of account creation.

2. User Profile Service (UserDB)

Purpose: Stores PII (Personally Identifiable Information) and KYC compliance data.

Table: UserProfiles

Field Name	Data Type / Constraints	Description
Id	PK	Unique Profile ID.
AuthUserId	Unique, GUID (FK Logical)	Links to AuthDB.Users.Id.
FullName	Varchar	User's legal name.
DateOfBirth	Date	Required for age-restricted services.
Address/City/State	Varchar	Location details.
PinCode	Varchar	Postal code.

Table: KycSubmissions

Field Name	Data Type / Constraints	Description
Id	PK	Unique KYC record ID.
AuthUserId	Unique, GUID	Links to the specific user.
DocumentType	Varchar	e.g., Passport, SSN, PAN.
Status	Enum (Default: Pending)	Pending, Approved, Rejected.
RejectionReason	Text	Notes if KYC fails.

3. Wallet Service (WalletDB)

Purpose: Manages financial balances and transaction history.

Table: Wallets

Field Name	Data Type / Constraints	Description
Id	PK	Unique Wallet ID.
AuthUserId	Unique, GUID	Owner of the wallet.
Balance	Decimal (18, 2)	Current available funds.
IsActive	Boolean	Status of the wallet.

Table: LedgerEntries

Field Name	Data Type / Constraints	Description
Id	PK	Transaction record ID.
WalletId	FK (Wallets.Id)	Reference to the parent wallet.
Type	Enum	CREDIT or DEBIT.
Amount	Decimal (18, 2)	Transaction value.
ReferenceId	GUID / String	External ID (e.g., OrderId, StripeTxnId).

4. Rewards Service (RewardsDB)

Purpose: Gamification engine for points and redemptions.

Table: RewardAccounts

Field Name	Data Type / Constraints	Description
AuthUserId	Unique (PK/FK)	Links user to rewards.
TotalPoints	Integer	Accumulated loyalty points.
Tier	Varchar (Default: Silver)	Loyalty level (Silver, Gold, Platinum).

Table: RewardCatalog

Field Name	Data Type / Constraints	Description
Title/Description	Varchar	Display details of the reward item.
PointsCost	Integer	Points required to redeem.
Stock	Int (Default -1)	Inventory count (-1 for infinite).

5. Notification & Admin Services

Purpose: System communications and audit logging.

Table: Notifications (NotificationDB)

Field Name	Data Type / Constraints	Description
AuthUserId	GUID	Recipient of the message.
Title/Message	Varchar/Text	Content of the alert.
IsRead	Boolean	Tracking for UI indicators.

Table: AdminActions (AdminDB)

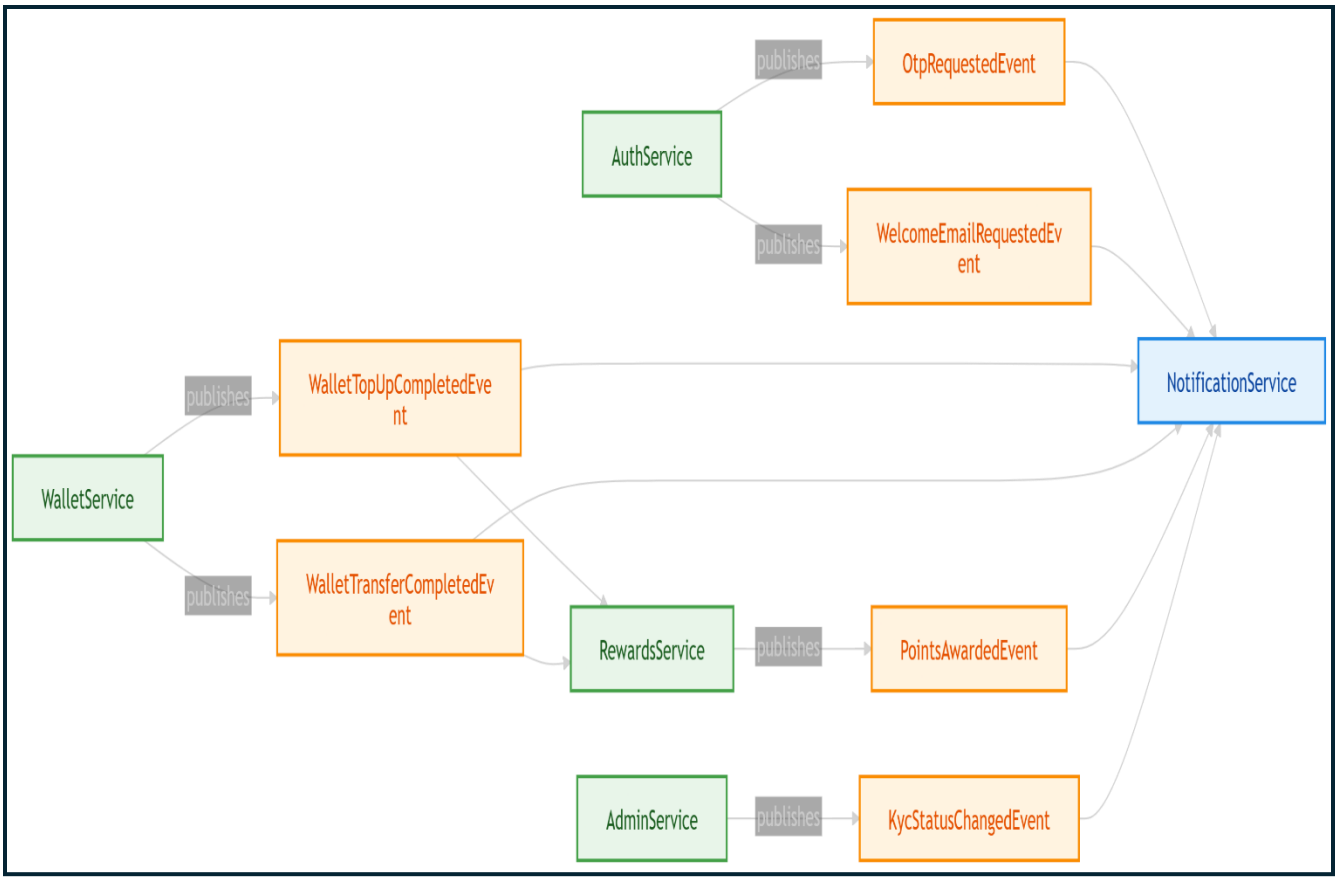
Field Name	Data Type / Constraints	Description
AdminAuthUserId	GUID	The administrator who took action.
ActionType	Varchar	e.g., BAN_USER, APPROVE_KYC.
TargetUserId	GUID	The user affected by the action.

7.4 Caching Usage

Service	Cached Data	Strategy	Eviction
AuthService	Refresh tokens, OTPs, reset tokens	IMemoryCache (in-process)	10 min (OTP), configurable (token)
WalletService	Wallet balance	IMemoryCache (in-process)	Short duration, cleared on mutation
RewardsService	Reward catalog	IMemoryCache (in-process)	5 minutes sliding

8. Integration Architecture

8.1 RabbitMQ Event Flow



8.2 Inter-Service HTTP Calls

AdminService uses typed HttpClients to call UserService and AuthService for administrative data aggregation. The AuthTokenHandler DelegatingHandler automatically propagates the caller's JWT Bearer token in outgoing requests, ensuring downstream services can authorize the forwarded request.

8.3 Downstream Service Communication

Calling Service	Called Service	Purpose
AdminService	UserService	Fetch pending KYC, review KYC
AdminService	AuthService	Fetch user list, toggle user status
AdminService	AuthService	Fetch email by AuthUserId for dashboard

9. Security Architecture

9.1 JWT Authentication

All secured endpoints require a valid JWT Bearer token issued by AuthService. The JWT payload contains NameIdentifier (AuthUserId), Email, and Role claims. Each service independently validates the JWT using a shared signing key configured in appsettings.json.

9.2 Role-Based Authorization

- User role: access to profile, wallet, rewards, notification endpoints.
- Admin role: access to all admin endpoints in AuthService, UserService, and AdminService.
- AdminService requires Admin role on all its endpoints.

9.3 Password Hashing

All user passwords are stored as BCrypt hashes. Plain-text passwords are never stored. BCrypt.Verify is used during login to compare the submitted password against the stored hash.

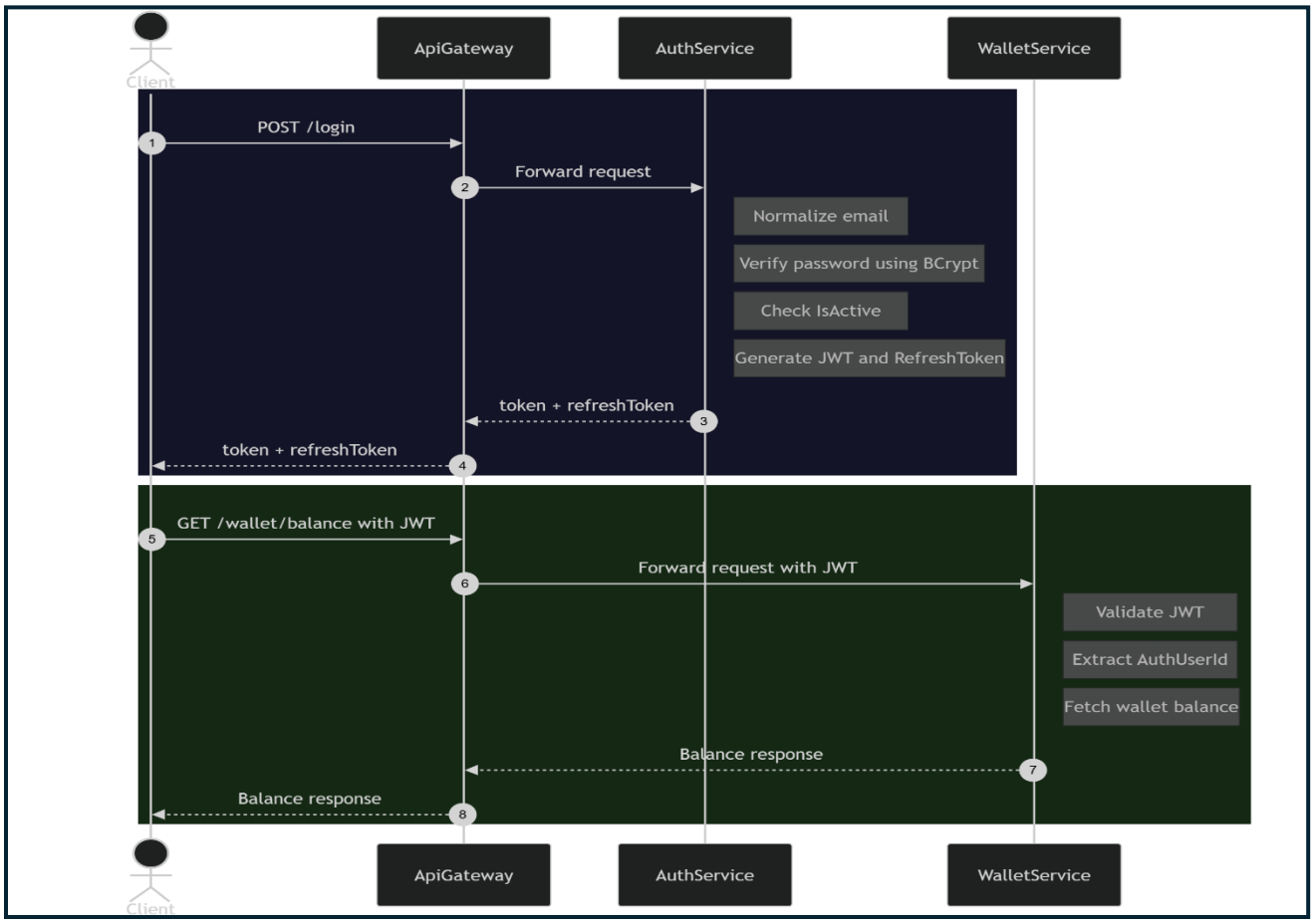
9.4 Token Forwarding

AdminService uses a typed HttpClient with AuthTokenHandler (a DelegatingHandler) to forward the caller's JWT to downstream services. This ensures that downstream service authorization checks receive the admin user's claims.

9.5 CORS Policy

ApiGateway is configured with a CORS policy permitting requests from http://localhost:4200 (the Angular development server). All methods and headers are allowed under this policy.

9.6 Authentication Flow Diagram



10. Non-Functional Requirements

Category	Requirement	Current Implementation
Scalability	Each microservice can be scaled independently.	Separate deployable ASP.NET Core processes.
Availability	Services should degrade gracefully under dependency failure.	TBD — no circuit breaker implemented yet.
Maintainability	Bounded contexts ensure changes in one service do not affect others.	Achieved via database-per-service and event contracts.
Performance	High-read data should be served from cache.	IMemoryCache on balance, catalog, and tokens.

Category	Requirement	Current Implementation
Security	No plain-text credentials. Tokens expire.	BCrypt hashing, JWT with expiry, OTP 10-min cache.
Logging	Request/error logging per service.	TBD — structured logging not explicitly configured.
Monitoring	Service health and metrics.	TBD — health check endpoints not confirmed in codebase.

11. Deployment View

11.1 Service Ports

Service	Port	Database	Notes
ApiGateway	5000	None	Ocelot routing only
AuthService	5001	AuthDB (SQL Server)	JWT issuer
UserService	5002	UserDB (SQL Server)	Profile + KYC
WalletService	5003	WalletDB (SQL Server)	Ledger operations
RewardsService	5004	RewardsDB (SQL Server)	Points + catalog
NotificationService	5005	NotificationDB (SQL Server)	In-app notifications
AdminService	5006	AdminDB (SQL Server)	Admin governance
RabbitMQ	5672 / 15672	N/A	AMQP + Management UI

11.2 Runtime Dependencies

- SQL Server instance (one logical instance, separate databases per service)
- RabbitMQ server with default AMQP port 5672
- Shared JWT signing key in appsettings.json across all services
- Admin secret key for admin registration (configured in AuthService appsettings)
- Google OAuth Client ID (configured in AuthService for Google login)

12. Current Implementation Status

12.1 Completed Backend Services

Service	Status	Notes
AuthService	Complete	All auth flows implemented
UserService	Complete	Profile and KYC flows implemented
WalletService	Complete	All wallet operations implemented
RewardsService	Complete	Points, catalog, redemption implemented
NotificationService	Complete	Storage and retrieval implemented
AdminService	Complete	KYC review, user management, dashboard
ApiGateway	Complete	Ocelot routes configured
Shared.Events	Complete	Event contracts defined

12.2 Remaining Work

- Angular frontend integration (planned, not yet implemented)
- End-to-end and integration test coverage
- Production hardening and deployment pipeline (CI/CD)
- Structured logging and distributed tracing (TBD)
- Health check endpoints (TBD)
- Circuit breaker / resilience policies for inter-service HTTP calls (TBD)

12.3 Angular Integration Scope

The Angular frontend is planned to consume all services through the ApiGateway. It should implement the following feature modules: Auth, User, Wallet, Rewards, Notification, Admin, and shared layout/navigation. The Angular app must manage JWT storage, refresh token flow, role-based route guards, and reactive HTTP communication via HttpClient with Interceptors.

13. Risks and Limitations

Risk / Limitation	Impact	Recommended Improvement
IMemoryCache is in-process only	Cache not shared across instances; invalidation limited	Replace with Redis for distributed caching
No circuit breaker for HTTP calls	AdminService dependent services may cascade-fail	Implement Polly retry/circuit-breaker policies
No distributed tracing	Cross-service request debugging is difficult	Integrate OpenTelemetry or Application Insights

Risk / Limitation	Impact	Recommended Improvement
No health checks implemented	Gateway cannot detect unhealthy services	Add ASP.NET Core health check middleware
OTP and token are in-memory only	Lost on service restart; not HA-ready	Migrate to Redis with TTL support
No integration tests confirmed	Risk of regression during feature additions	Add xUnit integration test projects
CORS origin hardcoded to :4200	Production deployment requires reconfiguration	Parameterize CORS origin via configuration
Admin secret key in appsettings	Accidental exposure in source control	Move to environment variable or secrets vault

14. Conclusion

ZyntraPay implements a clean microservices architecture that effectively separates authentication, user management, wallet operations, rewards, notifications, and admin governance into independently deployable services. The use of Ocelot as an API gateway, RabbitMQ for asynchronous event processing, JWT for stateless authentication, and the database-per-service pattern establishes a solid foundation for a scalable and maintainable digital wallet platform.

The back-end implementation is functionally complete across all seven services. The primary remaining work consists of the Angular frontend integration, comprehensive testing, and production hardening. The risks identified in Section 13 are addressable without structural changes to the existing architecture.

— End of High-Level Design Document —